upscale ai

INTERNAL · CONFIDENTIAL

ENGINEERING LEARNING BRIEF · ~5 MIN

Apple Silicon ML: From Cluster Bottlenecks to On-Chip Routing

Motivation + software map + a 60-second live demo — not a deep dive. Companion PDF has the full study plan.

Upscale AI builds open, high-performance AI networking (SkyHammer™ scale-up, scale-out Ethernet) so GPU clusters run efficiently at datacenter scale — upscaleai.com. This session is about the *other* end of the stack: what happens on a single Apple SoC when you prototype or run models locally.

upscale ai

INTERNAL · CONFIDENTIAL

OPEN WITH THIS · ~90 SEC

The standup that lost the room — then won it back

> "We spent six months making GPU-to-GPU faster across the rack. Then my demo on a MacBook stuttered — and I realized I was still thinking PCIe copies on a chip that doesn't have them."

① **Cluster world**
Bottleneck = network between accelerators (what we build at Upscale)

② **Laptop world**
Same tensors — but CPU, GPU & ANE share one memory pool (UMA)

③ **Insight**
You don't "move data to the GPU" — you *pick an engine* and measure

Ask the room: "Who has run a model on a Mac and assumed it behaved like a CUDA box?" — that gap is why we're here, not because Apple replaces datacenter networking.

Story bridges public mission → personal tooling gap

upscale ai

INTERNAL · CONFIDENTIAL

CONTEXT · ~45 SEC

Why this matters for us

What we do (public)

- Pure-play **AI networking** — silicon, systems, software
- **Scale-up** (SkyHammer™) + **scale-out** open Ethernet
- Open standards: UEC, UALink, SONiC

What this talk is

- **On-device literacy** — Mac/iOS prototyping & inference
- Metal / MLX / Core ML — when to use which
- ANE vs GPU — routing, not "faster shaders"

What it isn't

- Replacing cluster/network design knowledge
- Apple datacenter playbooks or roadmaps
- A mandate to ship on Mac — a **skill multiplier**

Analogy: **scale-out fabric** (Upscale) vs **scale-up on one die** (Apple SoC + UMA). Different layer — same discipline: find the real bottleneck, instrument, fix the right layer.

Source: upscaleai.com, public press (Series A, Jan 2026)

upscale ai

INTERNAL · CONFIDENTIAL

HARDWARE · ~45 SEC

One SoC, three engines, one memory pool

Engines

- **CPU** — control & small ops
- **GPU** — parallel math (Metal, MLX)
- **ANE** — compiled inference subgraphs

UMA

CPU, GPU, ANE reference the **same RAM**.
Placement is scheduling — not a host↔device copy.

Takeaway

Apple ML = **systems routing** + Instruments — not "port CUDA and hope."

upscale ai

INTERNAL · CONFIDENTIAL

SOFTWARE · ~60 SEC

What to use when

```
Prototype / train on Mac  →  MLX (Python / Swift)
Ship inference on device  →  Core ML + Xcode reports
Custom GPU kernel         →  Metal (+ MPS)
From PyTorch, etc.          →  coremltools → .mlpackage

        ▼
    CPU  |  GPU  |  ANE   (unified memory)
```

Goal	Start here
Explore LLMs locally	**MLX**
iOS / macOS app	**Core ML**
Low-level GPU	**Metal**

> **ANE ≠ faster GPU SMs.** It runs supported inference ops — often in parallel — so the GPU does the rest.

Full phased plan → Apple-Silicon-ML-Study-Guide.pdf

upscale ai

INTERNAL · CONFIDENTIAL

PATH · ~45 SEC

Phased self-study (~3–4 mo, a few hrs/week)

Order

- **0–1** UMA mental model + stack map
- **2–4** Metal labs → Core ML + ANE instruments
- **5–7** MLX + Swift mini-app → ANE vs GPU

Weekly: read → lab → instrument

Capstone: ANE + MLX pipeline

Next step for the team

Pick one buddy pair, run the **closing demo** together this week, then open the PDF for Phase 0 exit checkpoint.

upscale ai

INTERNAL · CONFIDENTIAL

CLOSE STRONG · LIVE DEMO ~2–3 MIN (4 ACTS)

Try it now: lazy graph → GPU wakes up → CPU contrast

What you're watching

read first

- **Activity Monitor → GPU History** — the "instrument"
- **Act 1:** build tensors → graph stays **flat** (lazy)
- **Act 2:** `mx.eval` → graph **spikes** (real work)
- **Act 3:** same API on **CPU** — slower wall clock

Run

paced

```
# once: uv sync ./demo.sh # test run (no pauses): ./demo.sh --quick
```

Why it matters

the value

- **Not CUDA:** no `.cuda()` / PCIe copy — UMA + engine choice
- **Lazy MLX:** cheap until `mx.eval` — like building a graph, then launching
- **Systems tie-in:** pick GPU vs CPU ≈ pick engine; Core ML also picks ANE vs GPU

> **Say out loud:** "Flat → spike is the whole lesson. If you only see one blip, you missed act 1."

No uv?

Fallback: `system_profiler SPHardwareDataType | head -12` — install uv before Phase 5.